

beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is another solution to this [latex.org forum query](#).

Contents

1	Implementation	1
1.1	Package declarations	1
1.2	Overview	1
1.3	Main scanner	2
1.3.1	Grammar	2
1.3.2	Storage	5
1.3.3	Comma separated lists	10
1.3.4	Delimiters	12
1.3.5	Assignments, function calls and identifiers	13

1 Implementation

Identify the internal prefix ($\text{\LaTeX3}$ DocStrip convention).

```
1 <@@=bnvs>
```

1.1 Package declarations

```
2 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
3 \ProvidesExplPackage
4   {beanoves}
5   {2025/06/03}
6   {1.0}
7   {Named overlay specifications for beamer}
```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string **beanoves** or containing the case insensitive string **bnvs** delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the **\Beanoves** command to build a data model whereas the resolution translates queries into **beamer** specifications based on this data model.

The development is made easier due to a facility layer over **expl**.

```

8 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
9   \A \s* (?
10
11     • 2 → V
12     ( [^:]+? )
13
14     • 3, 4, 5 → A : Z? or A :: L?
15     | (? ( [^:]+? ) \s* : (? \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
16
17     • 6, 7 → ::(L:Z)?
18     | (? :: \s* (? ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
19
20     • 8, 9 → :(Z::L)?
21     | (? : \s* (? ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
22   )
23   \s* \Z
24 }
```

1.3 Main scanner

We use the same scanner to parse definitions and queries. We do not use any key–value facility built into **L^AT_EX**.

1.3.1 Grammar

Hereafter is a pragmatic grammar used for parsing, not all the expressions covered do have a real meaning. The characters al have category other. We start with basic entities.

1 $\langle space \rangle$	::= Horizontal white space characters
2 $\langle lower \rangle$	::= a lowercase letter
3 $\langle upper \rangle$	::= an uppercase letter
4 $\langle digit \rangle$	::= a decimal digit
5 $\langle sign \rangle$	::= '-' '+'
6 $\langle comma \rangle$	::= $\langle space \rangle$ ',' $\langle space \rangle$
7 $\langle equal \rangle$	::= $\langle space \rangle$ ('=' '?=' '"=' '+=') $\langle space \rangle$
8 $\langle alpha \rangle$	::= $\langle lower \rangle$ $\langle upper \rangle$ '_'
9 $\langle alnum \rangle$	::= $\langle alpha \rangle$ $\langle digit \rangle$
10 $\langle index \rangle$	::= $\langle sign \rangle^* \langle digit \rangle^+$
11 $\langle number \rangle$	::= $\langle sign \rangle^? \langle significand \rangle \langle exponent \rangle^?$
12 $\langle significand \rangle$	::= ('.'? $\langle digit \rangle^+$) ($\langle digit \rangle^+$ '.' $\langle digit \rangle^*$)
13 $\langle exponent \rangle$	::= ('e' 'E') $\langle sign \rangle^? \langle digit \rangle^+$
14 $\langle other \rangle$	::= $\langle number \rangle$ '+' '-' '*' '/'

`\c__bnvs_index_regex` Signed integer.

(End of definition for \c__bnvs_index_regex.)

```
17 \regex_const:Nn \c__bnvs_index_regex {
18   [-+]* \d+
19 }
```

`\c__bnvs_dot_index_regex` Signed integer with a dot . just before.

(End of definition for \c__bnvs_dot_index_regex.)

```
20 \regex_const:Nn \c__bnvs_dot_index_regex {
21   \. \ur{c__bnvs_index_regex}
22 }
```

`\c__bnvs_A_dot_index_Z_regex` Full signed integer with a dot . jus before.

(End of definition for \c__bnvs_A_dot_index_Z_regex.)

```
23 \regex_const:Nn \c__bnvs_A_dot_index_Z_regex {
24   \A \ur { c__bnvs_dot_index_regex } \Z
25 }
```

`\c__bnvs_A_index_Z_regex` Signed integer with no capture groups.

(End of definition for \c__bnvs_A_index_Z_regex.)

```
26 \regex_const:Nn \c__bnvs_A_index_Z_regex { \A \ur{c__bnvs_index_regex} \Z }
```

`\c__bnvs_dot_R_regex` A possibly empty list of . $\langle \text{short name} \rangle$ items representing a path. Integers are allowed as well.

```
27 \regex_const:Nn \c__bnvs_R_regex {
28   \ur{c__bnvs_index_regex} | [[:alnum:]]+
29 }
```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_dot_R_regex` A possibly empty list of . $\langle \text{short name} \rangle$ items representing a path. Integers are allowed as well.

```
30 \regex_const:Nn \c__bnvs_dot_R_regex {
31   \. \ur{c__bnvs_R_regex}
32 }
```

(End of definition for \c__bnvs_dot_R_regex.)

Then we define identifiers. $\langle S \rangle$ stands for a short identifier, $\langle P \rangle$ stands for a path component and $\langle R \rangle$ stands for a relative component.

```
15 \langle S \rangle      ::= \langle alpha \rangle \langle alnum \rangle*
16 \langle P \rangle      ::= \langle digit \rangle* \langle S \rangle
17 \langle R \rangle      ::= ( ' . ' \langle P \rangle ) | ( ' . ' \langle index \rangle ) | \langle N \rangle
18 \langle N \rangle      ::= '[' \langle Rhs \rangle ']'
19 \langle IKSPR \rangle ::= ( ( \langle S \rangle? '!' )? \langle S \rangle ) | ' . ' \langle P \rangle ) \langle R \rangle*
20 \langle Rhs \rangle     ::= \langle Cr1 \rangle | \langle Sqr \rangle | \langle VAZL \rangle | \langle VAZLRnd \rangle
```

`\c__bnvs_S_regex` This regular expression is used for short names. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit.

```

33 \regex_const:Nn \c__bnvs_S_regex {
34   [[[:alpha:]]_][[:alnum:]]_*
35 }

```

(End of definition for `\c__bnvs_S_regex`.)

`\c__bnvs_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

36 \regex_const:Nn \c__bnvs_P_regex {
37   \d*[[[:alpha:]]_][[:alnum:]]_*
38 }

```

(End of definition for `\c__bnvs_P_regex`.)

`\c__bnvs_dot_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

39 \regex_const:Nn \c__bnvs_dot_P_regex {
40   \. \ur{c__bnvs_P_regex}
41 }

```

(End of definition for `\c__bnvs_dot_P_regex`.)

The lists between curly delimiters are used for $\langle \text{key} \rangle = \langle \text{value} \rangle$ definitions, with optional values. Each $\langle \text{CrLI} \text{tm} \rangle$ end with either a comma, an unbalanced closing delimiter or `\q_stop`. Here $\langle \text{Rhs} \rangle$ ends exactly the same.

```

21 <CrI>      ::= '{' <CrLIst> '}'
22 <CrLIst>   ::= <space> <CrLItm> ( <comma> <CrLItm> )* <space>
23 <CrLItm>   ::= <CrLEql> | <IKSPR>
24 <CrLEql>   ::= ( <IKSPR> <equal> )+ <Rhs>

```

The lists between square delimiters are used for $\langle \text{key} \rangle = \langle \text{value} \rangle$ definitions, with optional keys.

```

25 <Sqr>      ::= '[' <SqrLst> ']'
26 <SqrLst>   ::= <space> <SqrItm> ( <comma> <SqrItm> )* <space>
27 <SqrItm>   ::= <SqrEql> | <Rhs>
28 <SqrEql>   ::= ( ( <IKSPR> | <index> ) <equal> )+ <Rhs>

```

For individual ranges or values:

```

29 <VAZL>     ::= <Blnc> | ( <Blnc> ':' <Blnc>? ( ':' <Blnc>? )? )
30             | ( <Blnc> ':' <Blnc>? ( ':' <Blnc>? )? )
31 <Blnc>     ::= ( <space> | <IKSPR> | <Rnd> | <Sqr> | <CrI> | <Rlv> | <Raw> |
   ⇐ <other> ) *
32 <VAZLRnd>  ::= '(' <VAZLLst> ')'
33 <VAZLLst>  ::= <space> <VAZL> ( <comma> <VAZL> )* <space>
34 <Rlv>      ::= '?' <VAZLRnd>

```

For function calls,

```

35 <Raw>      ::= ' ' '(' <RawLst> ')'
36 <RawLst> ::= ( <space> | <Rlv> | <RawRnd> | <RawSqr> | <RawCrl> | <RawOtr>
    ↪ ) *
37 <RawRnd> ::= '(' <RawLst> ')'
38 <RawSqr> ::= '[' <RawLst> ']'
39 <RawCrl> ::= '{' <RawLst> '}'
40 <RawOtr> ::= <comma> | <alnum> | <other>

```

1.3.2 Storage

There are different policies to store the intermediate results of a scanning operation. One possibility is to feed a global variable as scanning occurs. As you cannot use unbalanced braces in function arguments, we can use other characters than usual “\”, “{” and “}” before we rescan the final result with an appropriate catcode regime. Another possibility is to heavily use $\TeX$ groups to store intermediate results in local variables and only consider properly balanced material. At the end of the group, the scanned material must be forwarded to the group owner. The latter is used here.

There is also a special situation for raw material that is stored separately and exported at once before other material is exported or when a scan group ends.

```

\BNVS_scan_store:n      \BNVS_scan_store:n {<something>}
\BNVS_scan_xprt_stored: \BNVS_scan_xprt_stored:

```

`\BNVS_scan_store:n` synchronously cumulates its argument in the `scan_stored` variable. The function `\BNVS_scan_xprt_stored:` is executed both at the beginning of the function `\BNVS_scan_xprt_raw:n` and when a scan group ends. It exports the contents of the stored `<material>` as `\:n{<material>}` instruction and cleans the variable.

¶ At the top level, `\BNVS_scan_xprt_stored:` is a do nothing operation. It is redefined by `\BNVS_scan_store:n` when there is something to do.

```

42 \cs_new:Npn \BNVS_scan_xprt_stored: {
43   \__bnvs_tl_if_empty:cF { scan_stored } {
44     \BNVS_tl_use:nv {
45       \__bnvs_tl_clear:c { scan_stored }
46       \BNVS_scan_xprt_raw:n { \:n }
47       \BNVS_scan_xprt_crl:n
48     } { scan_stored }
49   }
50 }

51 \cs_new:Npn \BNVS_scan_store:n #1 {
52   \tl_if_empty:nF { #1 } {
53     \__bnvs_tl_put_right:cn { scan_stored } { #1 }
54   }
55 }

```

<code>\BNVS_scan_xprt_reset:</code>	<code>\BNVS_scan_xprt_reset:</code>
<code>\BNVS_scan_xprt_raw:n</code>	<code>\BNVS_scan_xprt_raw:n {<something>}</code>
<code>\BNVS_scan_xprt_crl:n</code>	<code>\BNVS_scan_xprt_crl:n {<something>}</code>
<code>\BNVS_scan_use:n</code>	<code>\BNVS_scan_use:n {<code:n>}</code>

`\BNVS_scan_xprt_reset:` resets the scan storage, called when starting a new $\TeX$ group dedicated to scanning. `\BNVS_scan_xprt_<raw or crl>n` feeds the scan storage with `<something>`, as is in the `raw` version, enclosed between one pair of curly braces in the `crl` version. Implementation detail: the second is based on the first. `\BNVS_scan_use:n` puts the scanned material enclosed in one pair of braces after the instructions `<code:n>`.

```

56 \cs_new:Npn \BNVS_scan_xprt_reset: {
57   \__bnvs_tl_clear:c { scanned }
58 }

59 \cs_new:Npn \BNVS_scan_xprt_raw:n #1 {
60   \BNVS_scan_xprt_stored:
61   \__bnvs_tl_put_right:cn { scanned } { #1 }
62 }

63 \cs_new:Npn \BNVS_scan_xprt_crl:n #1 {
64   \BNVS_scan_xprt_stored:
65   \BNVS_scan_xprt_raw:n { { #1 } }
66 }

67 \cs_new:Npn \BNVS_scan_use:n #1 {
68   \BNVS_scan_xprt_stored:
69   \BNVS_tl_use:nv { #1 } { scanned }
70 }

```

<code>\BNVS_begin_scan:N</code>	<code>\BNVS_begin_scan:N <function:n></code>
<code>\BNVS_begin_scan:n</code>	<code>\BNVS_begin_scan:n {<body:n>}</code>
<code>\BNVS_end_scan:</code>	<code>\BNVS_end_scan: <input stream></code>
<code>\BNVS_end_scan:n</code>	<code>\BNVS_end_scan:n {<prepare:e>} <input stream></code>

`\BNVS_begin_scan:N` sets `\BNVS_scan_ended:n` to `<function:n>`, opens a $\TeX$ scan group and calls `\BNVS_scan_xprt_reset:`. Similarly, `\BNVS_begin_scan:n` sets the body of `\BNVS_scan_ended:n` to `<body:n>`, opens a $\TeX$ scan group and calls the functions `\BNVS_scan_xprt_reset:`. The function `\BNVS_end_scan:` closes the $\TeX$ scan group and calls `\BNVS_scan_ended:nn` on the scanned material. Usage: call one of `\BNVS_begin_scan:N <f:n>` or `\BNVS_begin_scan:n {<body:n>}`, feed the `scanned` variable with some `<material>` and finally call `\BNVS_end_scan:`. At this point, the `scanned` has eventually been appended by the instruction `<f:n> {<material>}` or its `<body:n>` counterpart.

When provided, the `<after:>` code is executed last.

```

71 \cs_new:Npn \BNVS_begin_scan:N #1 {
72   \cs_set:Npn \BNVS_scan_ended:nn ##1 ##2 {
73     #1 { ##1 }
74     ##2
75   }
76   \BNVS_begin:
77   \cs_set_eq:NN \BNVS_scan_prepare:n \use:n
78   \BNVS_scan_xprt_reset:
79 }

```

```

80 \cs_new:Npn \BNVS_begin_scan:n #1 {
81   \cs_set:Npn \BNVS_scan_ended:nn ##1 ##2 {
82     #1
83     ##2
84   }
85   \BNVS_begin:
86   \cs_set_eq:NN \BNVS_scan_prepare:n \use:n
87   \BNVS_scan_xprt_reset:
88 }

89 \cs_new:Npn \BNVS_end_scan: {
90   \BNVS_scan_use:n {
91     \BNVS_end_scan:n
92   }
93 }

```

¶ We start by exporting the stored material, if any.

```

94 \cs_new:Npn \BNVS_end_scan:n #1 {
95   \exp_args:Nne \use:n {
96     \BNVS_end:
97     \BNVS_scan_ended:n
98   } {
99     \tl_if_empty:nTF { #1 } {
100       \BNVS_scan_prepare:n { }
101     } {
102       \BNVS_scan_prepare:n { \exp_not:n { #1 } }
103     }
104   }
105 }

```

<code>\BNVS_scan_ended:n</code>	<code>\BNVS_scan_ended:n {⟨scanned⟩}</code>
<code>\BNVS_scan_prepare:n</code>	<code>\BNVS_scanned:n {⟨scanned⟩}</code>

`\BNVS_scan_ended:n` is a utility function to store its argument in the `scanned` variable. Automatically set by `\BNVS_begin_scan:...` functions. Used by `\BNVS_end_scan:`. The default implementation just raises. `\BNVS_scan_prepare:n` is called before exporting. The argument is enclosed inside `\exp_not:n{⟨...⟩}`. The default implementation just expands to its argument as is.

```

106 \cs_new:Npn \BNVS_scan_ended:n #1 {
107   \BNVS_scan_ended:nn { #1 } { }
108 }

109 \cs_new:Npn \BNVS_scan_ended:nn #1 #2 {
110   \BNVS_error:n { Missing-definition-of-\BNVS_scan_ended:nn. }
111 }

112 \cs_new_eq:NN \BNVS_scan_prepare:n \use:n

```

Utility function to store the argument enclosed within one pair of balanced curly braces, after a $\TeX$ group is closed.

In the sequel, `<no code:w>` is expected to read the `<input stream>` until a `\q_stop` token that is gobbled. The `_scan_` in function names is a shortcut to indicate a peek and remove operation.

```
\_bnvs_scan_q_stop:F  \_bnvs_scan_q_stop:F {(no code:w)} <input stream>
```

On scanning a `\q_stop` from the head of the `<input stream>`, calls the `\BNVS_scanned_stop:` branch function, otherwise execute `<no code:w>`.

```
113 \BNVS_new:cpn { scan_q_stop:F } {
114   \peek_meaning_remove:NTF \q_stop {
115     \BNVS_scanned_stop:
116   }
```

⊘ There must be absolutely no code here. No hooking allowed either.

```
117 }
```

⚡ Do nothing operation at the top level.

```
118 \cs_set:Npn \BNVS_scanned_stop: {}
```

```
\_bnvs_scan_comma:Fw  \_bnvs_scan_comma:Fw <no code:w> <input stream>
\BNVS_scanned_comma:
```

On scanning a comma from the head of the `<input stream>`, calls `\BNVS_scanned_comma:` branch function, otherwise execute `<no code:w>`.

```
119 \BNVS_new:cpn { scan_comma:Fw } #1 {
120   \peek_remove_spaces:n {
121     \peek_charcode_remove:NTF , {
122       \peek_remove_spaces:n { \BNVS_scanned_comma: }
123     } {
124       #1
125     }
126   }
127 }
```

⚡ Do nothing operation at the top level.

```
128 \cs_set:Npn \BNVS_scanned_comma: {
129 }
```

```
\_bnvs_scan_colon:Fw  \_bnvs_scan_colon:Fw {(no code:w)} <input stream>
```

On scanning a standalone colon from the `<input stream>`, calls the branch function `\BNVS_scanned_one_colon:w`. On scanning multiple colons from the `<input stream>`, calls the `\BNVS_scanned_colons:w` branch function. In other cases, execute the `<no code:w>`.

```
130 \BNVS_new:cpn { scan_colon:Fw } #1 {
131   \peek_remove_spaces:n {
132     \peek_charcode_remove:NTF : {
133       \peek_charcode_remove:NTF : {
134         \peek_remove_spaces:n { \BNVS_scanned_colons:w }
135       } {
136         \peek_remove_spaces:n { \BNVS_scanned_one_colon:w }
137       }
138     } {
```

```

139         #1
140     }
141 }
142 }

```

```

\BNVS_scanned_close:N \BNVS_scanned_close:N <clositer> <input stream>

```

```

\BNVS_scanned_close:w \BNVS_scanned_close:N is a branch function to be redefined by \__bnvs_scan_delimited:NNnFw.

```

It is called after the closing delimiter `<clositer>` has just been scanned. After it manages eventual errors in balancing delimiters, it calls the branch function `\BNVS_scanned_close:w` used many times.

```

143 \cs_set:Npn \BNVS_scanned_close:N #1 {
144   \BNVS_error:n { Unexpected~delimiter~`#1' }
145   \BNVS_scanned_close:w
146 }

```

❗ Do nothing operation at the top level.

```

147 \cs_set:Npn \BNVS_scanned_close:w {
148 }

```

```

\__bnvs_scan_close:Fw \__bnvs_scan_close:Fw <no:w> <input stream>

```

When a closing delimiter is scanned, call `\BNVS_scanned_close:N` with that delimiter as argument. Otherwise execute `<no:w>` code.

```

149 \group_begin:
150 \char_set_catcode_group_begin:N <
151 \char_set_catcode_group_end:N >
152 \char_set_catcode_other:N \{
153 \char_set_catcode_other:N \}
154 \BNVS_new:cpn < scan_close:Fw > #1 <
155   \peek_charcode_remove:NTF%(
156   ) <
157   \BNVS_scanned_close:N%(
158   )
159 > <
160   \peek_charcode_remove:NTF%[
161   ] <
162   \BNVS_scanned_close:N%[
163   ]
164 > <
165   \peek_charcode_remove:NTF%{
166   } <
167   \BNVS_scanned_close:N %{
168   }
169 > <
170   #1
171 >
172 >
173 >
174 >
175 \group_end:

```

1.3.3 Comma separated lists

\BNVS_scan_csl_item:w **\BNVS_scan_csl_item:w** *<input stream>*

Scan a single item in a comma separated list. The item ends at the first top level comma, the first unbalanced delimiter if one is found, or when a **\q_stop** is found. The **scan_stored** variable is fed with material that cannot be processed by *<scan:Fw>* function.

```

176 \cs_set:Npn \BNVS_scan_csl_item:w {
177   \BNVS_scan_special:TFw {
178     \BNVS_scan_csl_item:w
179   } {
180     \__bnvs_scan_comma:Fw {
181       \__bnvs_scan_close:Fw {
182         \__bnvs_scan_q_stop:F {
183           \__bnvs_scan_csl_other:Nw
184         }
185       }
186     }
187   }
188 }

189 \cs_new:Npn \BNVS_scan_special:Fw {
190   \BNVS_error:n { \BNVS_scan_special:Fw-undefined-at-top-level. }
191 }

192 \BNVS_new:cpn { scan_csl_other:Nw } #1 {
193   \BNVS_scan_store:n { #1 }
194   \BNVS_scan_csl_item:w
195 }
```

__bnvs_scan_csl:nNnnw **__bnvs_scan_csl:nNnnw** *<{<xprt:n>} <special:Fw> <{<closed:w>} <{<stopped:w>}>*
<input stream>

Scan the *<input stream>* for a comma separated list. The list ends at the first unbalanced delimiter if one is found, or when **\q_stop** is found. The scanned material is braced and prefixed with the *<xprt:n>* function before it is exported. Each individual items ends at a top level comma, or at the end of the list. The *<closed:w>* code is executed when the closing delimiter is found, otherwise the *<stopped:w>* code is executed.

If the input stream reads **a,b,c\q_stop**, while **\csl:n** is the *<xprt:n>* function, the scanned material is **\csl:n{:\nn{a}{:\nn{b}{:\nn{c}{}}}}**. If the input stream only reads **\q_stop** the scanned material is simply **\csl:n{}** for an empty list. Empty items are discarded.

```

196 \BNVS_new:cpn { scan_csl:nNnnw } #1 #2 #3 #4 {
197   \BNVS_begin_scan:n {
❗ Export <xprt:n>{...}.
198     \BNVS_scan_xprt_raw:n { #1 { ##1 } }
199   }

200   \cs_set:Npn \BNVS_scan_special:Fw { #2 }
```

```

201
202   \cs_set:Npn \BNVS_scanned_close:N ##1 {
❧ Close the current scan group and forwards to \BNVS_scanned_close:N, then execute
  \closed:w code.
203     \BNVS_end_scan:
204     \BNVS_scanned_close:N ##1
205     #3
206   }
207   \cs_set:Npn \BNVS_scanned_stop: {
❧ Close the current scan group and forwards to \BNVS_scanned_stop:, then execute
  \stopped:w code.
208     \BNVS_end_scan:
209     \BNVS_scanned_stop:
210     #4
211   }
212   \BNVS_scan_csl:w
213 }
214 \cs_new:Npn \BNVS_scan_csl:w {
215   \BNVS_begin_scan:N \BNVS_scan_xprt_crl:n
216   \cs_set:Npn \BNVS_scanned_close:N ##1 {
❧ Close the current scan group and forwards to \BNVS_scanned_close:N, which is a re-
    \BNVS_end_scan:
    \BNVS_scanned_close:N ##1
    }
217   \cs_set:Npn \BNVS_scanned_stop: {
❧ Close the current scan group and forwards to \BNVS_scanned_stop:, which is a recursive
    \BNVS_end_scan:
    \BNVS_scanned_stop:
    }
218   \BNVS_begin_scan:N \BNVS_scan_xprt_raw:n
❧ On start, we do not know how many items there are, we always plan less than one.
219   \cs_set:Npn \BNVS_scan_prepare:n ##1 {
220     \tl_if_empty:nF { ##1 } {
221       \exp_not:N \:nn { ##1 } {}
222     }
223   }
224   \cs_set:Npn \BNVS_scanned_comma: {
225     \BNVS_scan_xprt_stored:
226     \__bnvs_tl_if_empty:cTF { scanned } {
227       \BNVS_scan_csl_item:w
228     } {
229   }
❧ After a comma, things change when something has been scanned.

```

```

235 \cs_set:Npn \BNVS_scan_prepare:n #####1 {
236   \tl_if_empty:nF { #####1 } {
237     \exp_not:N \:nn { #####1 }
238   }
239 }
240 \BNVS_end_scan:

```

¶ The second components of the `\:nn` function is just a csl, go recursive.

```

241 \BNVS_scan_csl:w
242 }
243 }
244 \BNVS_scan_csl_item:w
245 }

```

1.3.4 Delimiters

```

\__bnvs_scan_delimited:NNnFw \__bnvs_scan_delimited:NNnnFw <open> <close> {<open:nw>} {<after:w>}
\__bnvs_scan_delimited:NNnnFw {<no:w>} <input stream>

```

```

\__bnvs_scan_delimited:NNnFw <open> <close> {<open:nw>} {<no:w>}
<input stream>

```

If we scan the `<open>` delimiter (amongst standard open delimiters), we open a group that will be closed on scanning the next unbalanced closing delimiter. Only the given `<close>` delimiter raises no error. The defined `\BNVS_scanned_close:N` branch function is then executed with the closing delimiter as argument. If the scanning is stopped prematurely, then the `\BNVS_scanned_stop:` branch is executed, after an error is raised. Uses `\__bnvs_scan_csl:w` to scan the list between the delimiters. Uses `<after:w>` or `\__bnvs_scan_blnc:w` to scan material after the closing delimiter. In all other cases, the `<no:w>` code branch is executed.

```

246 \BNVS_new:cpn { scan_delimited:NNnnFw } #1 #2 #3 #4 #5 {
247   \peek_charcode_remove:NTF #1 {
248     \BNVS_begin_scan:n {
249       \BNVS_scan_xprt_raw:n { #3 { ##1 } }
250     }
251     \cs_set:Npn \BNVS_scanned_close:N ##1 {
252       \tl_if_eq:nnF { #2 } { ##1 } {
253         \BNVS_error:n { Unexpected~`##1'~instead-of~`#2'. }
254       }
255       \BNVS_end_scan:

```

¶ After the closing material, we continue.

```

256   #4
257 }
258 \cs_set:Npn \BNVS_scanned_stop: {
259   \BNVS_error:n { Missing~`#2'. }
260   \BNVS_end_scan:
261   \BNVS_scanned_stop:
262 }

```

¶ Scan for a list.

```

263     \_bnvs_scan_csl:w
264 } {
265     #5
266 }

```

⊘ There must be absolutely no code here. No hooking allowed either.

```

267 }

268 \BNVS_new:cpn { scan_delimited:NNnFw } #1 #2 #3 {
269     \_bnvs_scan_delimited:NNnnFw #1 #2 #3 {
270         \_bnvs_scan_blnc:w
271     }

```

⊘ There must be absolutely no code here. No hook is allowed either.

```

272 }

```

1.3.5 Assignments, function calls and identifiers

\c__bnvs_IKS_regex The start of a qualified dotted name, with three capture groups.

(End of definition for \c__bnvs_IKS_regex.)

```

273 \regex_const:Nn \c__bnvs_IKS_regex {

    1: or the optional frame <id> with
    2: the exclamation mark

274     (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?

    3: followed by a non empty short name.

275     ( \ur{c__bnvs_S_regex} )
276 }

```

\c__bnvs_IKSP_regex A qualified dotted name is the qualified name of an overlay set possibly followed by a dotted path. Four capture groups. Called by [_bnvs_scan_IKSP:TTFw](#).

(End of definition for \c__bnvs_IKSP_regex.)

```

277 \regex_const:Nn \c__bnvs_IKSP_regex {

    1: or the optional frame <id> with
    2: the exclamation mark

278     (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?

    3: followed by a non empty short name.

279     ( \ur{c__bnvs_S_regex} )

    4: the non empty component of the dotted alternative

280     | \. ( \ur{c__bnvs_P_regex} )
281 }

```

```

\__bnvs_scan_IKSP:TTFw \__bnvs_scan_IKSP:TTFw {<IS body:nnw>} {<ISP body:nnnw>} {<no:w>} <input stream>

```

Scan the *<input stream>* for an identifier. if one is found, execute the *<ISP body:nnnw>* code or the *<IS body:nnw>* instructions depending on the match, with the contents of I, S and P matches as arguments. Otherwise execute *<no:w>* code. Implementation detail: the P, I and S variables are set locally.

```

282 \cs_new:Npn \BNVS_scanned_IP_IKSP:www #1 #2 #3 #4 #5 #6 {
283   \tl_if_empty:nTF { #6 } {

```

❗ Not a “*<P>*”:

```

284   \tl_if_empty:nTF { #4 } {

```

❗ No frame identifier given, use the current value store in the I variable.

```

285     \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
286     \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #5 }
287     \__bnvs_tl_set:cn S { #5 }
288   } {

```

❗ Frame identifier given. Proceed as before mutatis mutandis.

```

289     \__bnvs_tl_set:cn I { #3 }
290     \__bnvs_tl_clear_new:c { S(#3) }
291     \__bnvs_tl_set:cn { S(#3) } { #5 }
292     \__bnvs_tl_set:cn S { #5 }
293   }
294   \cs_set:Npn \BNVS:w ##1 ##2 { #1 }
295   \BNVS_tl_use:Nvv \BNVS:w I S
296 } {

```

❗ A “*<P>*”:

```

297   \__bnvs_tl_if_exist:cTF { S( \BNVS_tl_use:c I ) } {
298     \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I ) }
299   } {
300     \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
301     \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } X
302     \__bnvs_tl_set:cn { S } X
303   }
304   \cs_set:Npn \BNVS:w ##1 ##2 ##3 { #2 }
305   \BNVS_tl_use:Nvv \BNVS:w I S { #6 }
306 }
307 }

```

```

308 \BNVS_new:cpn { scan_IKSP:TTFww } #1 #2 #3 {
309   \peek_regex_replace_once:NnTF \c__bnvs_IKSP_regex {
310     { \1 } { \2 } { \3 } { \4 }
311   } {
312     \BNVS_scanned_IP_IKSP:www { #1 } { #2 }
313   } {

```

Execute *<no:>* code.

```

314     #3
315   }
316 }

```

```

\__bnvs_scan_ISRCQ:TFw \__bnvs_scan_ISRCQ:TFw {<after:w>} {<no:w>} <input stream>

```

Scan an identifier, a function call or an assignment then execute *<after:w>* or execute *<no:w>*. Starting with an empty *scanned* variable, here are some examples of the expected output for standalone identifiers

- $I!S \rightarrow \backslash ISR:w\{I\}\{S\}\{\}$
- $I!S.A \rightarrow \backslash ISR:w\{I\}\{S\}\{\backslash PR:w\{A\}\{\}\}$
- $I!S.A.B \rightarrow \backslash ISR:w\{I\}\{S\}\{\backslash PR:w\{A\}\{\backslash PR:w\{B\}\{\}\}\}$
- $I!S.+123 \rightarrow \backslash ISR:w\{I\}\{S\}\{\backslash NR:w\{-123\}\{\}\}$
- $I!S[1+2] \rightarrow \backslash ISR:w\{I\}\{S\}\{\backslash nR:w\{1+2\}\{\}\}$

and not standalone

- $I!S(\langle \dots \rangle) \rightarrow \backslash ISR:wn\{I\}\{S\}\{\}\{\langle \dots \rangle\}$
- $I!S?=\langle Rhs \rangle \rightarrow \backslash ISR:wnn\{I\}\{S\}\{\}\{?\}\{\langle Rhs \rangle\}$

As we do not want to overscan the same input, we must keep track of the identifier before we can know whether it is standalone or not.

```

317 \BNVS_new:cpn { scan_ISRCQ:TFw } #1 #2 {
318   \peek_regex_replace_once:NnTF \c__bnvs_IKS_regex { { \1 } { \2 } { \3 } } {

```

¶ We begin a scan group, the true branch will be executed at the end. By default we start on a standalone identifier, and we will change when necessary. *\BNVS_end_scan_IKS_RCQ:ww* will feed the *scanned* variable.

```

319   \BNVS_begin_scan:n {
320     \BNVS_scan_xprt_raw:n { ##1 }
321     #1
322   }
323   \cs_set:Npn \BNVS_scan_prepare:n ##1 {
324     \exp_not:N \ISR:w ##1
325   }
326   \BNVS_end_scan_IKS_RCQ:ww
327 } {
328   \peek_charcode_remove:NTF . {
329     \peek_regex_replace_once:NnTF \c__bnvs_P_regex { { \0 } } {

```

¶ We begin a scan group, the true branch will be executed at the end. By default we start on a standalone identifier, and we will change when necessary. *\BNVS_end_scan_P_RCQ:ww* will feed the *scanned* variable.

```

330   \BNVS_begin_scan:n {
331     \BNVS_scan_xprt_raw:n { ##1 }
332     #1
333   }
334   \cs_set:Npn \BNVS_scan_prepare:n ##1 {
335     \exp_not:N \IKS:w { ##1 }
336   }
337   \BNVS_end_scan_P_RCQ:ww
338 } {
339   \peek_regex_replace_once:NnTF \c__bnvs_index_regex { { \0 } } {

```

- ¶ We begin a scan group, the true branch will be executed at the end. By default we start on a standalone identifier, and we will change when necessary. The `scanned` variable will be properly fed by `\BNVS_end_scan_index_RCQ:ww`.

```

340         \BNVS_begin_scan:n {
341             \BNVS_scan_xprt_raw:n { ##1 }
342             #1
343         }
344         \cs_set:Npn \BNVS_scan_prepare:n ##1 {
345             \exp_not:N \IKS:w { ##1 }
346         }
347         \BNVS_end_scan_index_RCQ:ww
348     } {

```

- ¶ Choose the false branch and inject back the removed dot.

```

349         #2
350         .
351     }
352 }
353 } {

```

- ¶ Choose also the false branch and inject back the removed dot.

```

354         #2
355         .
356     }
357 }
358 }

359 \cs_new:Npn \BNVS_end_scan_IKS_RCQ:ww #1 #2 #3 {
360     \tl_if_empty:nTF { #2 } {

```

- ¶ No frame identifier given, use the current value, store in the I variable.

```

361         \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
362         \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #3 }
363     } {

```

- ¶ Frame identifier given. Proceed as before mutatis mutandis.

```

364         \__bnvs_tl_set:cn I { #1 }
365         \__bnvs_tl_clear_new:c { S(#1) }
366         \__bnvs_tl_set:cn { S(#1) } { #3 }
367     }

```

- ¶ Export the frame identifier and the short name.

```

368     \BNVS_tl_use:Nv \BNVS_scan_xprt_crl:n I
369     \BNVS_scan_xprt_crl:n { #3 }

```

- ¶ Scan next relative components, then a call or an assignment if any.

```

370     \BNVS_begin_scan:n {
371         \BNVS_scan_xprt_crl:n { ##1 }
372         \BNVS_end_scan_CQ:w
373     }
374     \BNVS_end_scan_R:w
375 }

376 \cs_new:Npn \BNVS_end_scan_R:w {

```

```

377 \peek_charcode_remove:NnTF . {
378   \peek_regex_replace_once:NnTF \c__bnvs_P_regex { { \0 } } {
379     \BNVS_end_scan_P:nw
380   } {
381     \peek_regex_replace_once:NnTF \c__bnvs_index_regex { { \0 } } {
382       \BNVS_end_scan_index:nw
383     } {
384       \BNVS_end_scan:

```

¶ No need to scan for a call or an assignment. But this is not at the right level.

```

385     \cs_set:Npn \BNVS_end_scan_CQ:w { \BNVS_end_scan: }
386     \STOP
387   .
388 }
389 }
390 } {
391   \peek_charcode_remove:NnTF [%]
392   {
393     \BNVS_begin_scan:n {
394       \BNVS_scan_xprt_raw:n { \NR:w { ##1 } }
395     }
396     \cs_set:Npn \BNVS_scanned_close:N ##1 {
397       \tl_if_eq:nnF { %[
398         ] } { ##1 } {
399         \BNVS_error:n { Unexpected~`##1'~instead-of~%[
400           `]' . }
401       }
402       \BNVS_end_scan:
403     }
404     \cs_set:Npn \BNVS_scanned_stop: {
405       \BNVS_error:n { Missing~%[
406         `]' . }
407       \BNVS_end_scan:
408       \BNVS_scanned_stop:
409     }

```

¶ Scan for a list.

```

410     \BNVS_begin_scan:n {
411       \BNVS_scan_xprt_crl: { ##1 }
412     }
413     \BNVS_end_scan_RhsLst:w
414   } {
415     \BNVS_end_scan:
416   }
417 }
418 }

```

¶ Exports `\PR:w{<key>}`, scan a relative component and calls `\BNVS_end_scan_R:w`.

```

419 \cs_new:Npn \BNVS_end_scan_P:nw #1 {
420   \BNVS_scan_xprt_raw:n { \PR:w { #1 } }
421   \BNVS_begin_scan:n {
422     \BNVS_scan_xprt_crl:n { ##1 }
423     \BNVS_end_scan_R:w
424   }
425   \BNVS_end_scan_R:w

```

```

426 }
❗ Exports \NR:w{<index>}, scan a relative component and calls \BNVS_end_scan_R:w.
427 \cs_new:Npn \BNVS_end_scan_index:nw #1 {
428   \BNVS_scan_xprt_raw:n { \NR:w }
429   \exp_args:Ne \BNVS_scan_xprt_crl:n { \int_eval:n { #1 } }
430   \BNVS_begin_scan:n {
431     \BNVS_scan_xprt_crl:n { ##1 }
432     \BNVS_end_scan_R:w
433   }
434   \BNVS_end_scan_R:w
435 }

436 \cs_new:Npn \BNVS_end_scan_CQ:w {
437   \peek_charcode_remove:NTF (%)
438   {
439     \BNVS_begin_scan:n {
440       \BNVS_scan_xprt_raw:n { ##1 }
441     }
442     \cs_set:Npn \BNVS_scanned_close:N ##1 {
443       \tl_if_eq:nnF { %(
444         ) } { ##1 } {
445         \BNVS_error:n { Unexpected~`##1'~instead~of~%(
446           `)''. }
447       }
448       \BNVS_end_scan:
449     }
450     \cs_set:Npn \BNVS_scanned_stop: {
451       \BNVS_error:n { Missing~%(
452         `)''. }
453       \BNVS_end_scan:
454       \BNVS_scanned_stop:
455     }

```